Code walkthrough

register_file.v

Register File

What it does	Four 4-bit registers R0-R3
Where it works	EX stage
Size	72 lines, explained in 6 small parts
Modules in this file	register_file
Calls (uses)	nothing (no other modules)
Called by	cpu_top.v - instance RF tb_register_file.v - instance DUT tb_cpu.v - reads CPU.RF.r0 ... CPU.RF.r3

The register file is the CPU's fast storage. It has two read ports (so both operands can be read at the same time) and one write port (for the result).

Map of the file

Part	Lines	What this part is
1	1-20	Header comment
2	21-31	Ports
3	32-33	The four registers
4	35-51	Write port (clocked)
5	53-61	Read port A (no clock)
6	63-72	Read port B (no clock)

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-20)

register_file.v lines 1-20

```
 1  // ======================================================================
 2  // FILE: register_file.v
 3  // 4 general-purpose registers (R0-R3), each 4 bits wide.
 4  //
 5  // Interface (as in the project plan):
 6  //   clk, rst, ra, rb, wa, wd, we, qa, qb
 7  //
 8  //   ra, qa : read port A  (address in, value out)  - no clock needed
 9  //   rb, qb : read port B  (address in, value out)  - no clock needed
10  //   wa, wd : write address and write data          - on clock edge
11  //   we     : write enable (1 = write)
12  //
13  // Register numbers: R0 = 00, R1 = 01, R2 = 10, R3 = 11
14  //
15  // CONNECTIONS
16  //   Calls       : nothing (no other modules)
17  //   Called by   : cpu_top.v - instance RF
18  //                 tb_register_file.v - instance DUT
19  //                 tb_cpu.v - reads CPU.RF.r0 ... CPU.RF.r3
20  // ======================================================================
```

- Lists the plan's interface: ra/qa and rb/qb are the two read ports, wa/wd/we the write port. Registers are numbered 00 to 11 in binary.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Ports (lines 21-31)

register_file.v lines 21-31

```
21  module register_file (
22      input            clk,
23      input            rst,
24      input      [1:0] ra,          // read address A
25      input      [1:0] rb,          // read address B
26      input      [1:0] wa,          // write address
27      input      [3:0] wd,          // write data
28      input            we,          // write enable
29      output reg [3:0] qa,          // read data A
30      output reg [3:0] qb           // read data B
31  );
```

- `ra`, `rb`: which registers to read (2 bits each).
- `wa`, `wd`, `we`: which register to write, the value, and the enable.
- `qa`, `qb`: the two values read. In the CPU, qa is Rd and qb is Rs.

Part 3 · The four registers (lines 32-33)

register_file.v lines 32-33

```
32      // the four registers
33      reg [3:0] r0, r1, r2, r3;
```

Four separate 4-bit registers `r0 ... r3`. Writing them separately (instead of an array) makes them easy to see in the waveform and in the testbench (`CPU.RF.r1`).

Part 4 · Write port (clocked) (lines 35-51)

register_file.v lines 35-51

```
35      // ---------------- WRITE (clocked) ----------------
36      always @(posedge clk) begin
37          if (rst) begin
38              r0 <= 4'b0000;        // reset clears all registers
39              r1 <= 4'b0000;
40              r2 <= 4'b0000;
41              r3 <= 4'b0000;
42          end
43          else if (we) begin
44              case (wa)
45                  2'd0: r0 <= wd;
46                  2'd1: r1 <= wd;
47                  2'd2: r2 <= wd;
48                  2'd3: r3 <= wd;
49              endcase
50          end
51      end
```

- At each rising clock edge: if reset, all four registers become 0.
- Otherwise, if we is 1, a case on wa picks which register receives wd. Only that one register changes.

Part 5 · Read port A (no clock) (lines 53-61)

register_file.v lines 53-61

```
53      // ---------------- READ PORT A (combinational) ----------------
54      always @(*) begin
55          case (ra)
56              2'd0: qa = r0;
57              2'd1: qa = r1;
58              2'd2: qa = r2;
59              2'd3: qa = r3;
60          endcase
61      end
```

always @(*) is combinational. A case on ra copies the chosen register to qa. When ra changes, qa changes immediately.

Part 6 · Read port B (no clock) (lines 63-72)

register_file.v lines 63-72

```
63      // ---------------- READ PORT B (combinational) ----------------
64      always @(*) begin
65          case (rb)
66              2'd0: qb = r0;
67              2'd1: qb = r1;
68              2'd2: qb = r2;
69              2'd3: qb = r3;
70          endcase
71      end
72  endmodule
```

Exactly the same as port A, using rb and qb. Having two ports lets ADD read Rd and Rs in the same cycle.

> **In short:** Read two registers at any time; write one register at the clock edge.